

Q.1 What is a Data Structure? Give 3 real-life examples.

A Data Structure is a way of organizing and storing data in a computer so that it can be accessed and modified efficiently.

Real-life Examples:

1. Array – Students sitting in a row of fixed seats.
2. Stack – Stack of plates in a canteen (Last plate kept is removed first).
3. Queue – People standing in a ticket booking line (First person enters first)

Q.2 Differentiate between Linear and Non-Linear Data Structures.

Linear Data Structure

Elements are arranged sequentially.

Easy to traverse.

Each element has one predecessor and one successor (except first and last).

Examples: Array, Stack, Queue, Linked List

Non-Linear Data Structure

Elements are arranged hierarchically or interconnected.

Traversal is more complex.

A node can have multiple connections.

Examples: Tree, Graph

Q.3 Explain the internal structure of an Array with a memory diagram.

An Array stores elements of the same data type in contiguous memory locations.

Example:

```
int arr[5] = {10,20,30,40,50};
```

Memory Diagram:

Index: 0 1 2 3 4
Array: | 10 | 20 | 30 | 40 | 50 |

Address:

1000 1004 1008 1012 1016

If an integer occupies 4 bytes:

Address of arr[i] = Base Address + (i × Size of Data Type)

Example:

arr[3] = 1000 + (3 × 4)
 = 1012

Q.4 Write the algorithm and C program for Array insertion.

Algorithm

1. Start.
2. Read array size n.
3. Input array elements.
4. Read position and element to insert.
5. Shift elements one position right from the insertion point.
6. Insert the new element.
7. Increase array size by 1.
8. Display the updated array.
9. Stop.

C Program

```
#include <stdio.h>

int main()
{
    int arr[100], n, pos, item;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
    for(int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    printf("Enter position to insert: ");
    scanf("%d", &pos);

    printf("Enter element: ");
    scanf("%d", &item);

    for(int i = n; i >= pos; i--)
        arr[i] = arr[i - 1];

    arr[pos - 1] = item;
    n++;

    printf("Array after insertion:\n");
    for(int i = 0; i < n; i++)
```

```
    printf("%d ", arr[i]);

    return 0;
}
```

Q.5 Explain Stack using LIFO concept with a diagram.

A Stack is a linear data structure that follows LIFO (Last In First Out).

The element inserted last is removed first.

Example

Push operations:

Push(10)

Push(20)

Push(30)

Stack Diagram

TOP

|

+----+

| 30 |

+----+

| 20 |

+----+

| 10 |

+----+

If Pop() is performed:

Removed = 30

Remaining Stack:

TOP

|

+----+

| 20 |

+----+

| 10 |

+----+

Q.6 What is the difference between Stack and Queue?

Stack

Follows LIFO (Last In First Out)
Insertion and deletion occur at the same end (Top).
Operations: Push, Pop
Example: Stack of plates

Queue

Follows FIFO (First In First Out)
Insertion at Rear and deletion at Front.
Operations: Enqueue, Dequeue
Example: Ticket counter line

Q.7 Write C code for Enqueue and Dequeue operations.

```
#include <stdio.h>
```

```
#define SIZE 5
```

```
int queue[SIZE];
```

```
int front = -1, rear = -1;
```

```
void enqueue(int item)
```

```
{  
    if(rear == SIZE - 1)  
    {  
        printf("Queue Overflow\n");  
        return;  
    }  
}
```

```
    if(front == -1)  
        front = 0;
```

```
    rear++;  
    queue[rear] = item;  
}
```

```
void dequeue()
```

```
{  
    if(front == -1 || front > rear)  
    {  
        printf("Queue Underflow\n");  
        return;  
    }  
}
```

```

    printf("Deleted Element: %d\n", queue[front]);
    front++;
}

```

```

int main()
{
    enqueue(10);
    enqueue(20);
    enqueue(30);

    dequeue();
    dequeue();
    return 0;
}

```

Q.8 Explain Singly Linked List with a node diagram.

A Singly Linked List is a collection of nodes where each node contains:

1. Data
2. Address of the next node

Node Structure

```

struct Node
{
    int data;
    struct Node *next;
};

```

Diagram

```

HEAD
|
v
+-----+-----+ +-----+-----+ +-----+-----+
| 10 | o---|--->| 20 | o---|--->| 30 | NULL |
+-----+-----+ +-----+-----+ +-----+-----+

```

Explanation

- HEAD points to the first node.
- Each node stores data and the address of the next node.
- The last node points to NULL, indicating the end of the list.

Advantages

- Dynamic size.
- Efficient insertion and deletion.
- Memory is allocated as needed.